Hermite Formulae

D. Renard, N. Desassis, X. Freulon

May 17th 2022

General Introduction

The objective of this document is to provide some guides for manipulation of Hermite polynomials. In particular, it will give some hints for calculating of complex expressions (e.g. metal above cutoff) and be double checked using a Monte Carlo approach.

It will also demonstrate the quality of the polynomial expansion for a given known distribution of the data (i.e. lognormal). Finally it will serve in demonstrating the ability of representing any function through a Hermite polynomial expansion.

Part I

The objective of the first part is to compute

$$\psi_1(\alpha, \beta) = \int \phi(\alpha + \beta\, u)\, g(u)\, du$$

and

$$\psi_2(\alpha, \beta) = \int \phi^2(\alpha + \beta\, u)\, g(u)\, du$$

for a function ϕ in order to compute the conditional expectation and the conditional variance, usually defined by its expansion in terms of Hermite polynomials:

$$\phi(y) = \sum_{n=0}^{N} a_n \eta_n(y)$$

In particular, we may be interested in the case where $\alpha = ry$ and $\beta = \sqrt{1 - r^2}$. Note that $|\beta| < 1$.

Many methods can be used to compute these draw.matrix:

- the Monte-Carlo integration (easy)
- the computation of the Hermite coefficient for ϕ and ϕ^2

$$\int_{-\infty}^{+\infty} \eta_n(\alpha + \beta\, u)\, g(u)\, du = (1 - \beta^2)^n\, \eta_n(\frac{\alpha}{\sqrt{1 - \beta^2}})$$

Evaluation on the lognormal case

Processing parameters

```
nbpoly = 100
nbsimu = 1000
params = list(nbpoly=nbpoly, nbsimu=nbsimu, yc=1.5)
```

We consider the Lognormal case (where the anamorphosis is known exactly).

```
m     <- 2.5
sigma <- 0.5
hn     = hermiteLognormal(m, sigma, params$nbpoly)
```

We calculate the various functions (expectation, ore and metal quantity, for expectation and standard deviation) for different values of y varying from -3 to 3 and different values of r varying from 0 to 1. The results are stored in a Db (organized as a matrix) whose parameters are given next.

```
y0  = -3
dy  = 0.1
dr  = 0.1
y   = seq(-3,3,by=dy)
ny  = length(y)
r   = seq(1,0,by=-dr)
nr  = length(r)
```

Comparisons

We evaluate the variable, the ore and metal quantity above a cutoff, in conditional expectation or in conditional standard deviation.

Calculating all elements using either Hermite expansion or Monte Carlo Simulations

```
draw.matrix(type=1,calcul=1,flag.est=TRUE,params,flag.cut=FALSE)
draw.matrix(type=2,calcul=1,flag.est=TRUE,params,flag.cut=FALSE)
draw.correlation(calcul=1,flag.est=TRUE,params)

draw.matrix(type=1,calcul=2,flag.est=TRUE,params)
draw.matrix(type=2,calcul=2,flag.est=TRUE,params)
draw.correlation(calcul=2,flag.est=TRUE,params)

draw.matrix(type=1,calcul=3,flag.est=TRUE,params)
draw.matrix(type=2,calcul=3,flag.est=TRUE,params)
draw.correlation(calcul=3,flag.est=TRUE,params)

draw.matrix(type=1,calcul=1,flag.est=FALSE,params,flag.cut=FALSE)
draw.matrix(type=2,calcul=1,flag.est=FALSE,params,flag.cut=FALSE)
draw.correlation(calcul=1,flag.est=FALSE,params)

draw.matrix(type=1,calcul=2,flag.est=FALSE,params)
draw.matrix(type=2,calcul=2,flag.est=FALSE,params)
draw.correlation(calcul=2,flag.est=FALSE,params)

draw.matrix(type=1,calcul=3,flag.est=FALSE,params)
draw.matrix(type=2,calcul=3,flag.est=FALSE,params)
draw.correlation(calcul=3,flag.est=FALSE,params)
```

Part II

We use the lognormal model to test some computations done with Hermite polynomials.

We consider the second order stationary model $Z_\lambda(x) = e^{\lambda Y(x) - \frac{1}{2}\lambda^2}$ where $Y(x)$ is a centered Gaussian model with autocorrelation ρ:

- $E\{Y\} = 0$
- $Cov(Y(x), Y(x+h)) = E\{Y(x)Y(x+h)\} = \rho(h)$
- $E\{Z_\lambda\} = 1$
- $Cov(Z_\lambda(x), Z_\lambda(x+h)) = E\{Z_\lambda(x)Z_\lambda(x+h)\} - 1 = e^{\lambda^2\rho(h)} - 1$

The anamorphosis ϕ_λ maps the Gaussian field into the lognormal SOS model $Z_\lambda = \phi_\lambda(Y) = \sum_{n=0}^{+\infty} \phi_n(\lambda) H_n(Y)$.

The Hermite coefficients are $\phi_n(\lambda) = \frac{(-\lambda)^n}{\sqrt{n!}}$ and we have

$$C_\lambda(h) = e^{\lambda^2\rho} - 1 = \sum_{n=1}^{+\infty} \phi_n^2(\lambda)\rho^n.$$

The recursion formula to compute the values of the Hermite polynomials are: $H_0(y) = 1$, $H_1(y) = -y$, and

$$H_{n+1}(y) = -\frac{1}{\sqrt{n+1}} y H_n(y) - \sqrt{\frac{n}{n+1}} H_{n-1}(y)$$

The next lines define some functions which can be explitely written in the lognormal case

```
GM_hermite <- function(y, nh){
  h <- matrix(NaN, nrow = length(y), ncol = 1+nh)
  h[,1] <- 1.0
  h[,2] <- -y
  for (i in 2:nh)
  {
    h[,1+i] <- -y*h[,i]/sqrt(i) - sqrt((i-1)/i)*h[,i-1]
  }
  h
}

# fonct
GM_LN_psi <- function(lambda, nh){
  (-lambda)^{0:nh} / sqrt(factorial(0:nh))
}
GM_LN_Cov_theo <- function(lambda){
  function(rho) {exp(lambda^2 * rho) - 1}
}
GM_LN_Cov_nh <- function(lambda, nh)
{
  psi <- GM_LN_psi(lambda = lambda, nh = nh)
  foo <- function(rho) {outer(X = rho, Y = 1:nh, FUN = function(r,n){r^n}) %*% psi[-1]^2}
}
```

The next paragraph initiates a data set and runs the previous functions. The output will serve as reference for comparison with gstlearn functions.

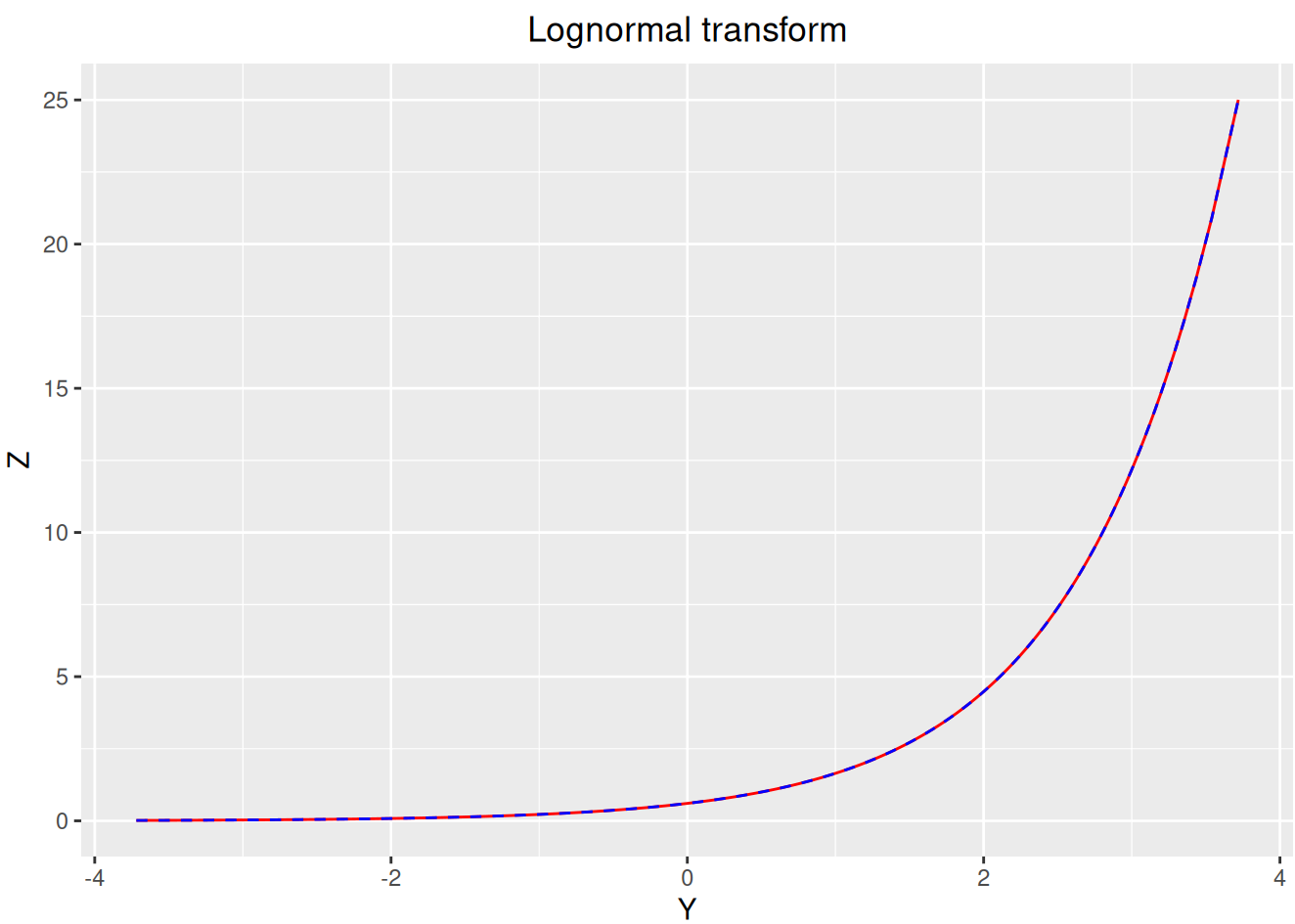
```
np <- 10000
nh <- 100
lambda <- 1.0
set.seed(123)
# anamorphosis in the lognormal case (theoretical)
yy <- qnorm(p = (1:np)/(np+1), mean = 0, sd = 1.0)
aymin = range(yy)[1]
aymax = range(yy)[2]
zz_theo <- exp(lambda*yy - lambda^2/2)
```

Perfoming the tests with the functions programmed internally

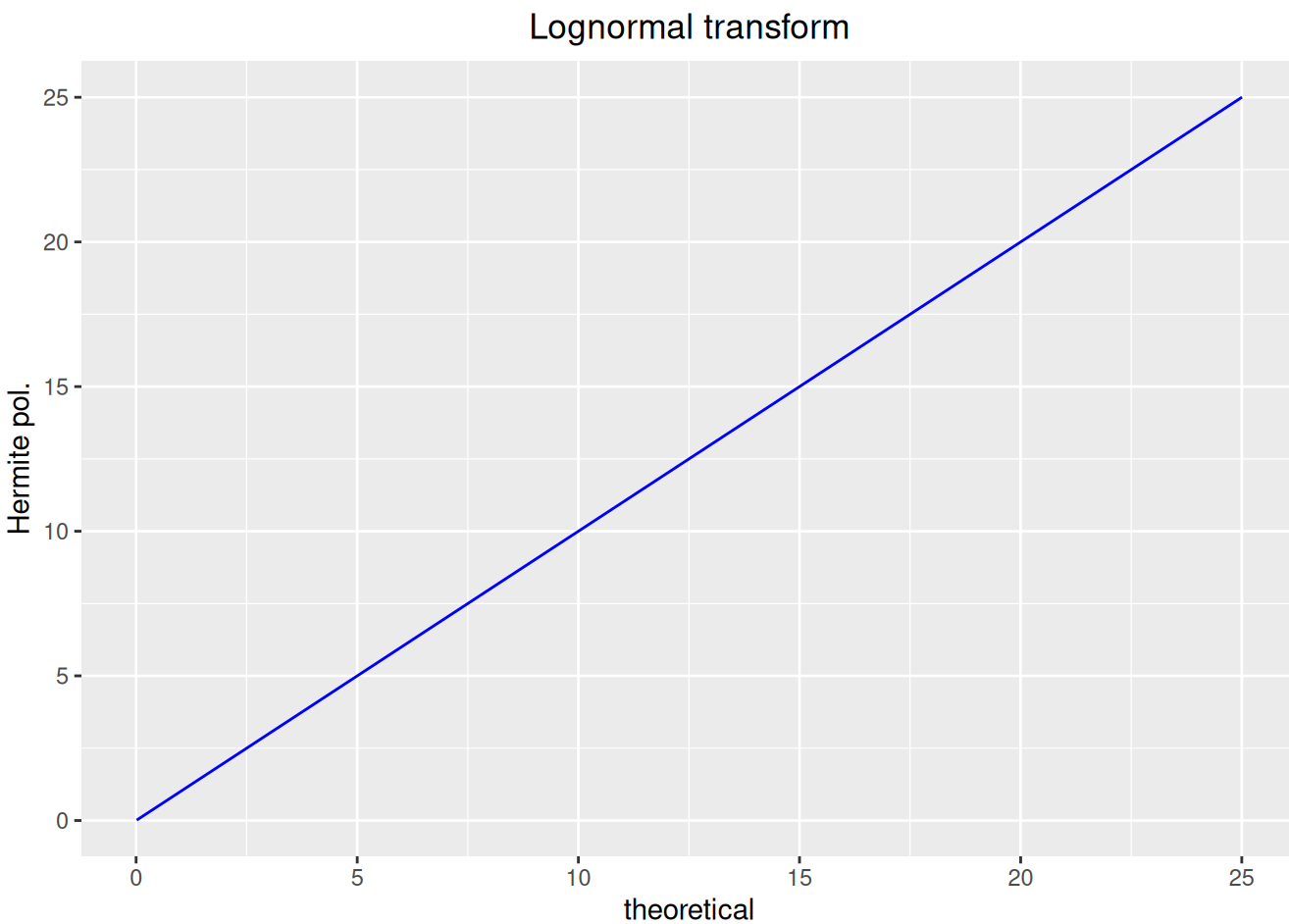
Anamorphosis in the lognormal case (Hermite pol.)

```
zz_hm   <- GM_hermite(y = yy, nh = nh) %*% GM_LN_psi(lambda = lambda, nh = nh)

p = plot.init()
p = p + plot.XY(yy, zz_theo, color = "red", flagLine=TRUE)
p = p + plot.XY(yy, zz_hm, color = "blue", flagLine=TRUE, linetype="dashed")
p = p + plot.decoration(xlab = "Y", ylab = "Z", title = "Lognormal transform")
plot.end(p)
```

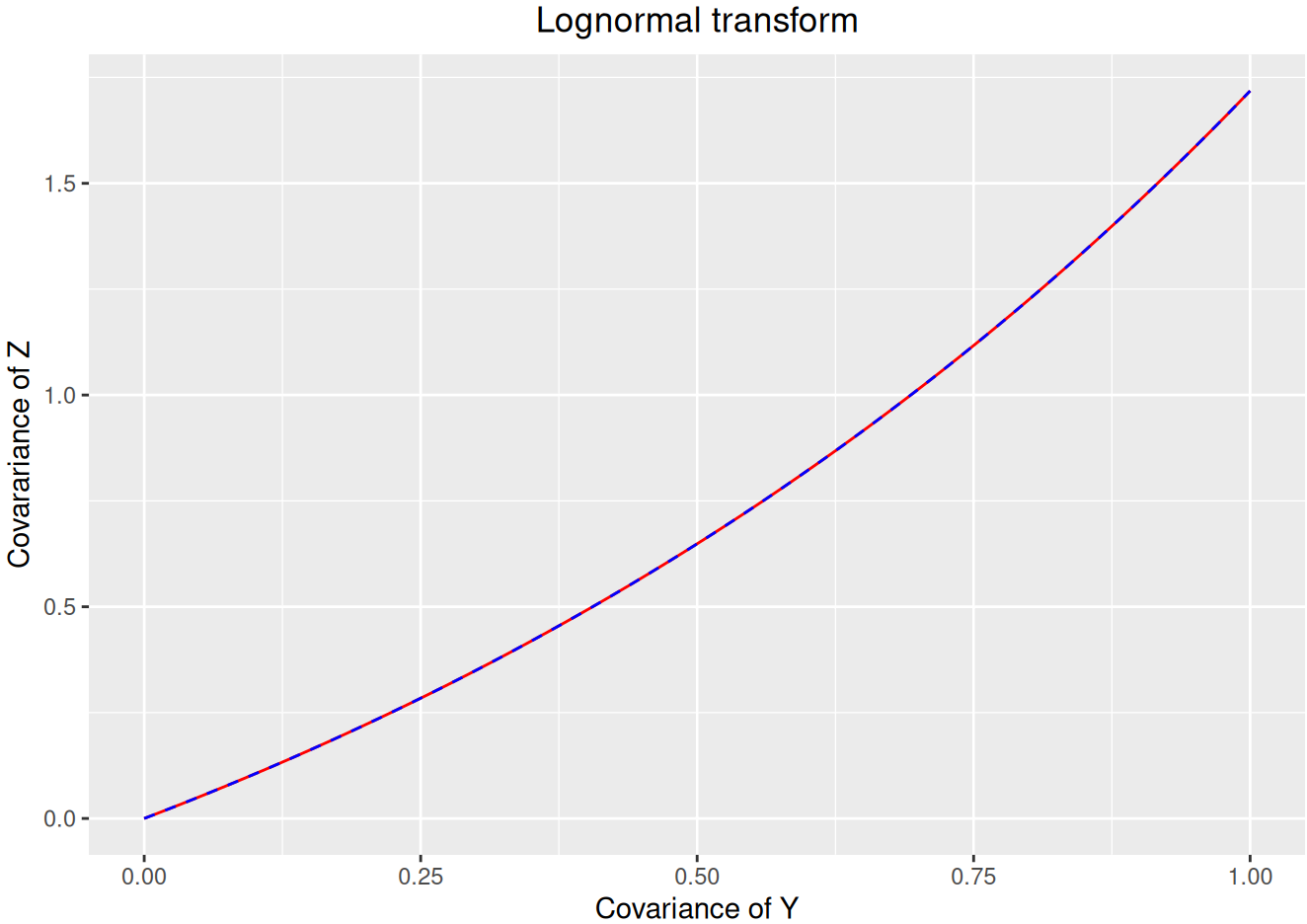


```
p = plot.init()
p = p + plot.XY(zz_theo, zz_hm, color = "blue", flagLine=TRUE)
p = p + plot.decoration(xlab = "theoretical", ylab = "Hermite pol.",
                        title = "Lognormal transform")
plot.end(p)
```



Covariance

```
r <- seq(from = 0, to =1, by = 0.01)
C_theo <- GM_LN_Cov_theo(lambda = lambda)
C_nh   <- GM_LN_Cov_nh(lambda = lambda, nh = nh)
p = plot.init()
p = p + plot.XY(r, C_theo(r), color = "red", flagLine=TRUE)
p = p + plot.XY(r,C_nh(r), color = "blue", linetype="dashed", flagLine=TRUE)
p = p + plot.decoration(xlab = "Covariance of Y", ylab = "Covarariance of Z",
                        title = "Lognormal transform")
plot.end(p)
```



Performing similar tests with gstlearn

```
db <- Db_createFromSamples(np,ELoadBy_SAMPLE(),zz_theo)
db$setNameByUID(db$getLastUID(),"zz_theo")
```

```
## NULL
```

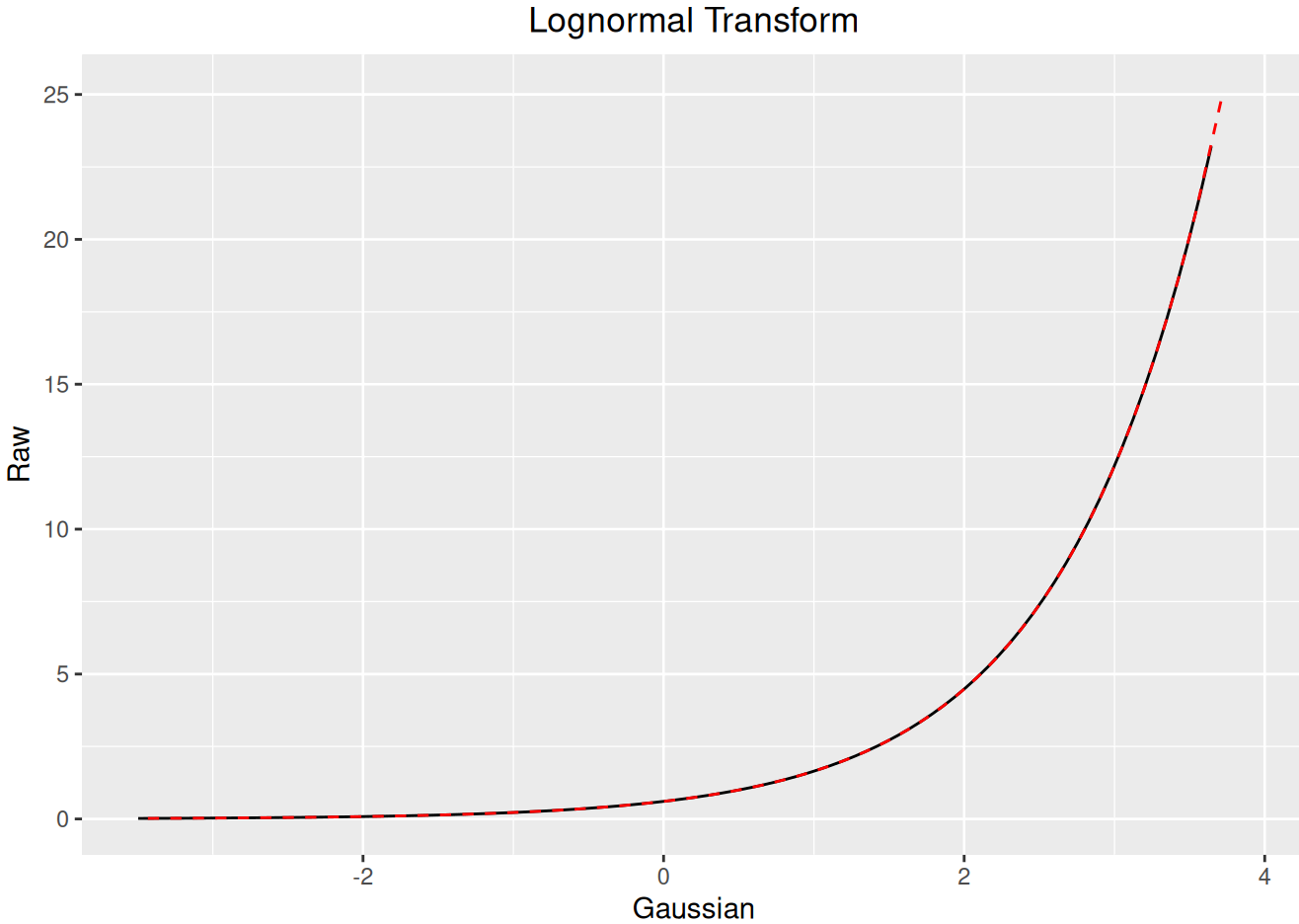
```
ana = AnamHermite_create(nh)
err = ana$fit(db, name="zz_theo")

# Patch the Hermite coefficients to have exact numerical values
localPsi <- GM_LN_psi(lambda = lambda, nh = nh-2)
err = ana$setPsiHns(localPsi)
ana$display()
```

```
##
## Hermitian Anamorphosis
## ----------------------
## Minimum absolute value for Y  = -3.5
## Maximum absolute value for Y  = 3.9
## Minimum absolute value for Z  = 0.0174079
## Maximum absolute value for Z  = 25.123
## Minimum practical value for Y = -3.1
## Maximum practical value for Y = 3.9
## Minimum practical value for Z = 0.0263251
## Maximum practical value for Z = 25.123
## Mean                          = 0.998127
## Variance                      = 1.63944
## Number of Hermite polynomials = 99
## Normalized coefficients for Hermite polynomials (punctual variable)
##                  [,  0]     [,  1]     [,  2]     [,  3]     [,  4]     [,  5]     [,  6]
##       [  0,]      1.000     -1.000      0.707     -0.408      0.204     -0.091      0.037
##       [  7,]     -0.014      0.005     -0.002      0.001      0.000      0.000      0.000
##       [ 14,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 21,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 28,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 35,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 42,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 49,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 56,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 63,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 70,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 77,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 84,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 91,]      0.000      0.000      0.000      0.000      0.000      0.000      0.000
##       [ 98,]      0.000
```

```
## NULL
```

```
p = plot.init()
p = p + plot.anam(ana, aymin=aymin, aymax=aymax)
p = p + plot.XY(yy, zz_theo, color="red", linetype="dashed", flagLine=TRUE)
p = p + plot.decoration(title="Lognormal Transform")
plot.end(p)
```




```
#legend("topleft", legend = c("theoretical", "gstlearn"), col = c("red", "black"), lty = c(1,1))
```

```
zz_rg = ana$gaussianToRawVector(yy)
p = plot.XY(zz_theo, zz_rg, flagLine=TRUE, color="blue")
p = plot.decoration(xlab="theoretical", ylab="Hermite pol.", title="Lognormal Transform")
plot.end(p)
#legend("topleft", legend = c("transform", "Y=X"), col = c("blue", "red"), lty = c(1,2))
```

Covariance

```
mod_y <- Model(nvar=1, ndim=2)
mod_y$addCovFromParam(ECov_SPHERICAL(),range=1,sill=1)
```

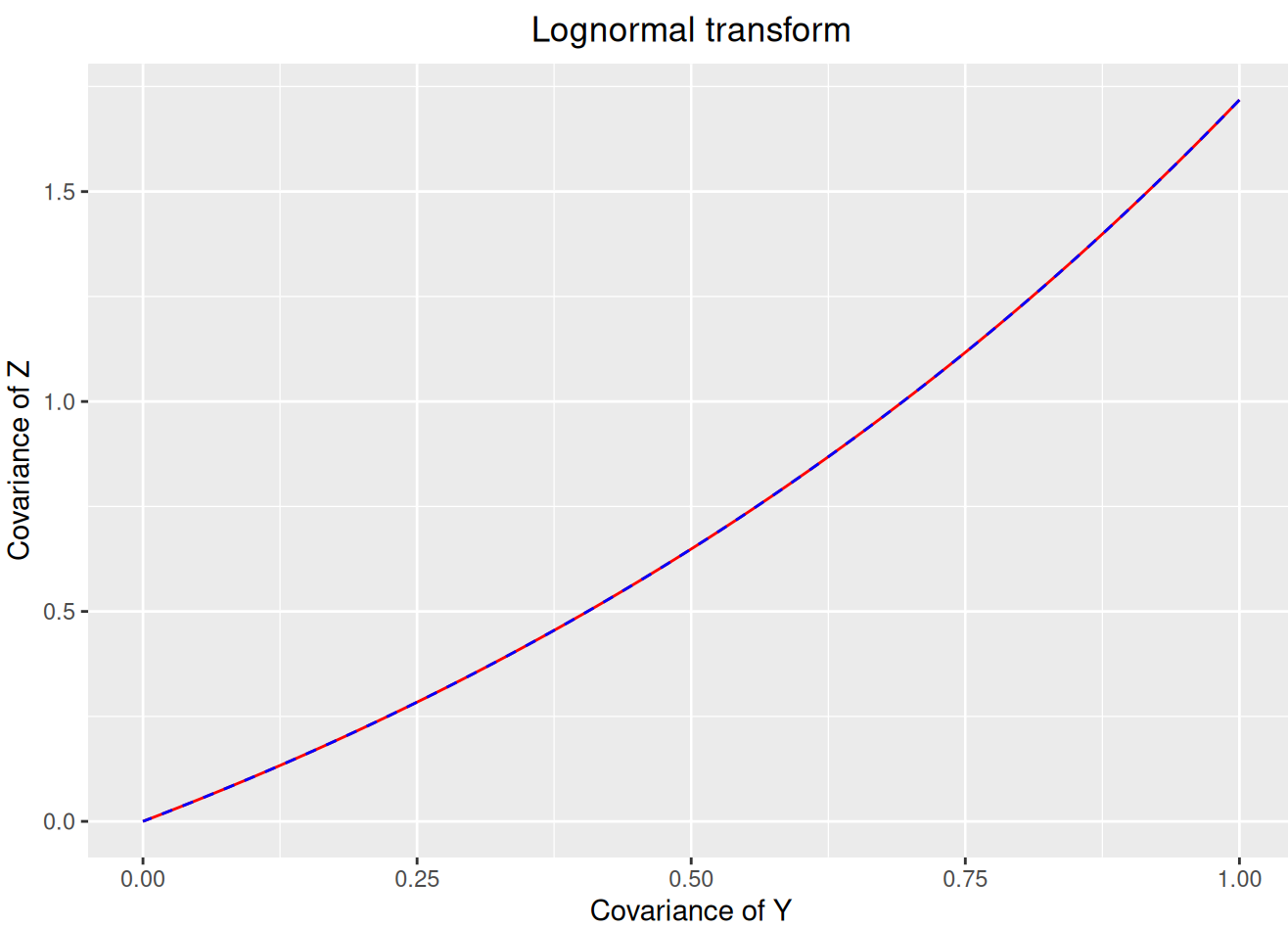
```
## NULL
```

```
h     <- seq(from = 0, to =1, by = 0.01)
r = mod_y$evalIvarNlag(h)
C_theo<- GM_LN_Cov_theo(lambda = lambda)
cov_theo<- C_theo(rho = r)

# Creation of the lognormal model (il faudrait pouvoir créer un modèle gaussien à partir de la covariance d
e la gaussienne mod_Y et de l'anamorphose gaussienne ana)
# mod_Z <- model.create.GM(model = mod_Y, anam = ana)
# cov_rg<- model.eval(model = mod_Z, h = h)

model.eval.GM <- function(model, anam, h){
  r = model$evalIvarNlag(h)
  psi <- anam$getPsiHns()
  nh <- length(psi) - 1
  outer(X = r, Y = 1:nh, FUN = function(rho,n){rho^n}) %*% (psi[-1])^2
}
cov_rg <- model.eval.GM(model = mod_y, anam = ana, h = h)

p = plot.init()
p = p + plot.XY(r, cov_theo, color = "red", flagLine=TRUE)
p = p + plot.XY(r, cov_rg, color = "blue", linetype = "dashed", flagLine=TRUE)
p = p + plot.decoration(xlab = "Covariance of Y", ylab = "Covariance of Z",
                        title = "Lognormal transform")
plot.end(p)
```




```
#legend("topleft", legend = c("theoretical", "gstlearn"), col = c("red", "blue"), lty = c(1,2))
```

The empirical anamorphosis does not reproduce the lognormal transform and the Hermite coefficient need to be altered.

Part III

The observed variable is

$$Y_{y_c} = \phi_{y_c}(Y) = Y \times 1_{Y \geq y_c} + y_c \times 1_{Y < y_c} = \sum_{n \geq 0} \phi_n(y_c) \times H_n(Y)$$

where normalized Hermite polynomials are $H_n(n) = \frac{1}{\sqrt{n!}} \frac{g^{(n)}(y)}{g(y)}$.

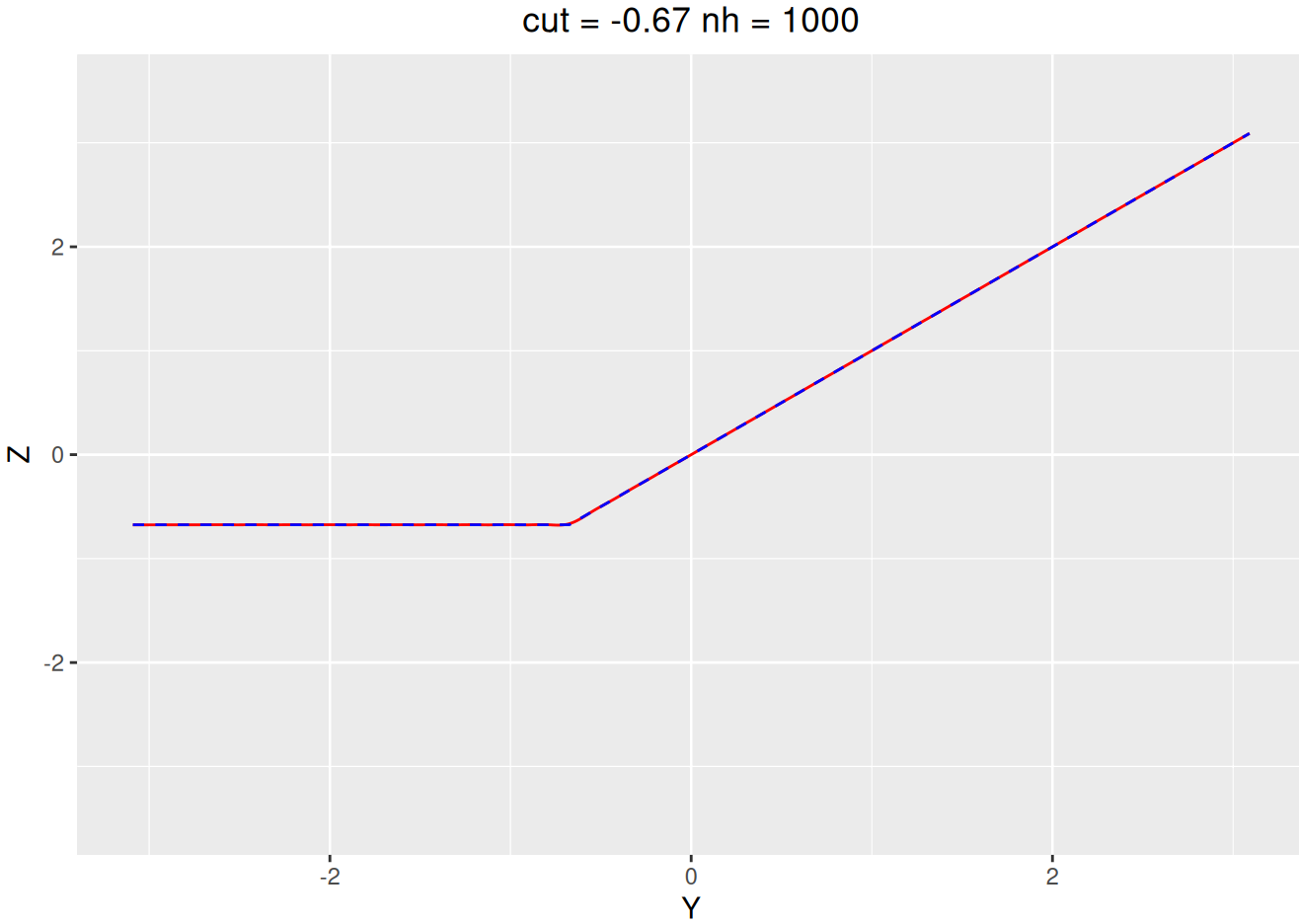
The computation of the coefficients $\phi_n(y_c)$ gives

- $\phi_0(y_c) = g(y_c) + y_c \times G(y_c)$,
- $\phi_1(y_c) = G(y_c) - 1$,
- $\phi_n(y_c) = g(y_c) \frac{H_{n-2}(y_c)}{\sqrt{n \times (n-1)}}$ for $n > 1$.

```
np <- 1000
nh <- 1000
yc <- qnorm(0.25)
yy <- qnorm(p = (1:np)/(np+1))
hm <- GM_hermite(y = yy, nh = nh-1)
```

Floor variable Z = max(yc, Y)

```
psi = hermiteCoefLower(yc, nh)
zz <- hm %*% psi
ylim = c(-3.5, 3.5)
p = plot.init()
p = p + plot.XY(yy, zz, color = "red", flagLine=TRUE)
p = p + plot.XY(yy, pmax(yc, yy), color = "blue", linetype = "dashed", flagLine=TRUE)
p = p + plot.decoration(xlab = "Y", ylab = "Z",
                        title = paste0("cut = ",round(yc,2), " nh = ", nh))
p = p + plot.geometry(ylim=ylim)
plot.end(p)
```

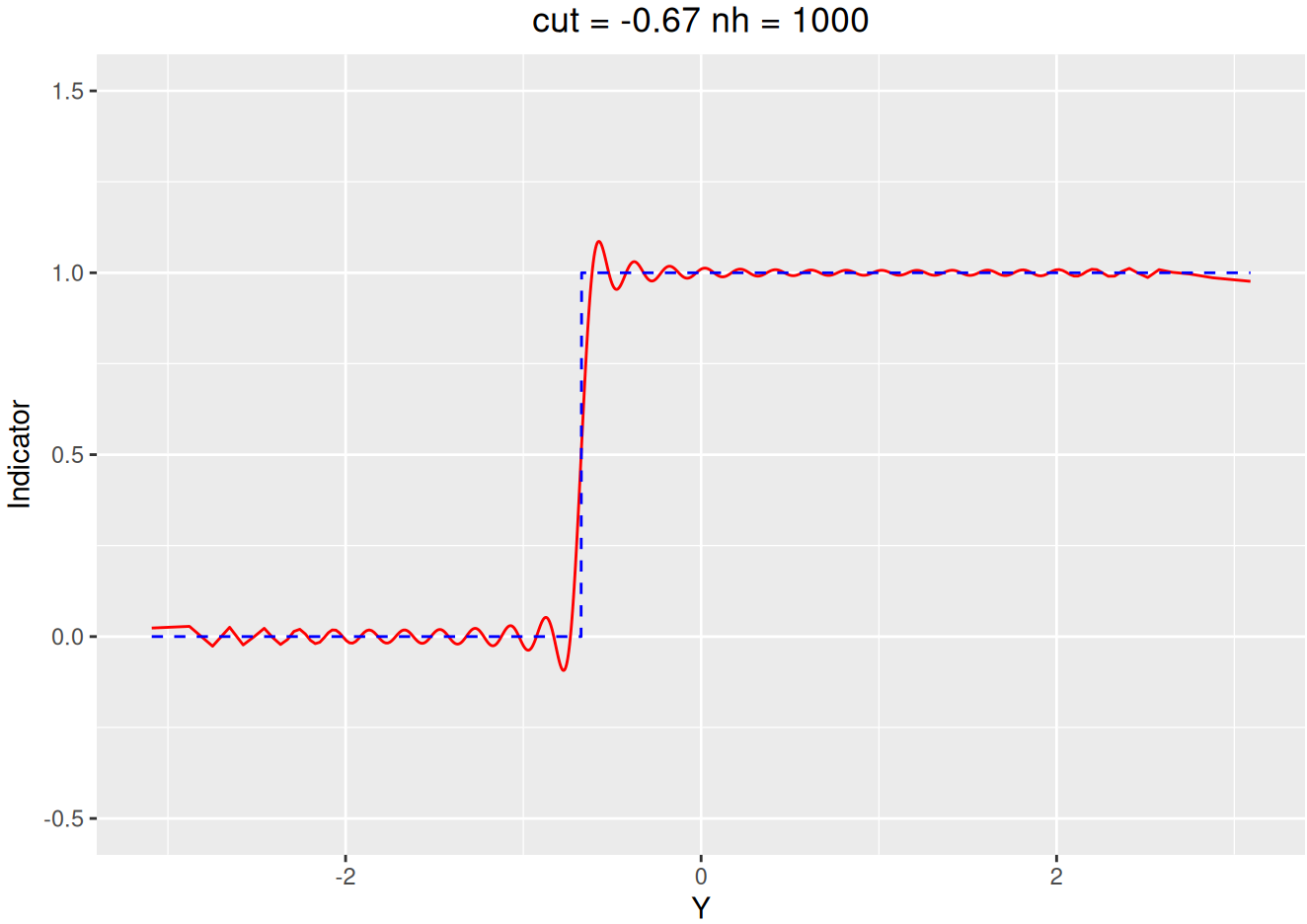



```
#legend("topleft", legend = c("theoretical", "Hermit. pol."), col = c("blue", "red"), lty = #c(1,2))
```

QC Indicator

```
psi = hermiteIndicatorLower(yc, nh)
zz <- hm %*% psi

p = plot.init()
p = p + plot.XY(yy, zz, color = "red", flagLine=TRUE)
p = p + plot.decoration(xlab = "Y", ylab = "Indicator",
                        title = paste0("cut = ",round(yc,2), " nh = ", nh))
p = p + plot.geometry(ylim = range(-0.5, 1.5))
p = p + plot.XY(yy, as.numeric(yy >= yc), color = "blue", linetype="dashed", flagLine=TRUE)
plot.end(p)
```




```
#legend("bottomright", legend = c("theoretical", "Hermit. pol."), col = c("blue", "red"), lty = #c(1,2))
```